```
class LocatorServiceSpec ... {
  ...

  "A Source" should {
    "be able to keep consuming from a failed and restored service" in {

      val clientSettings =      ...

      val client: LocatorService =          ❶
        LocatorServiceClient(clientSettings)

      val restartSettings = RestartSettings(
        minBackoff = 1.seconds,
        maxBackoff = 3.seconds,
        randomFactor = 0.2).withMaxRestarts(30, 3.minutes)
      val restartSource = RestartSource.withBackoff(restartSettings) {
                    ...
                  client.follow()           ❷
                    ...
      }

      val result =                          ❸
        restartSource
          .map(println)
          .run()

      Await.result(result, 90.seconds)      ❸
    }
  }
  ...
}
```

❶ Creates an instance of the gRPC service

❷ Creates a RestartSource consuming from the gRPC service

❸ Runs and prints responses from the service for 90 seconds

After you start the test, you can let the service run for a few seconds and then turn it off. The output of the test starts with a location with latitude 1 that is passed as input when the service is started. Once the service is stopped, the consumer finds the service `UNAVAILABLE`:

```
Location(1.0,-3.701101)
Location(2.0,-3.701101)
```